

# Proyecto Final: Juego de Totito

Inteligencia Artificial con el algoritmo Minimax · Java Swing

Nombre del estudiante: [ Escriba su nombre ]

Carne: [ Escriba su carne ]

Catedratico: [ Nombre del catedratico ]

Curso: Programacion II

Fecha de entrega: \_\_\_\_\_

## 1. Introduccion

El presente informe describe el desarrollo del **Juego de Totito** (tres en linea) implementado en el lenguaje **Java**, con una interfaz grafica construida exclusivamente con las librerias `javax.swing` y `java.awt`. El programa integra un agente de Inteligencia Artificial capaz de enfrentar al usuario tomando decisiones a traves del algoritmo de busqueda en arbol **Minimax**, garantizando un juego optimo e imbatible. Ademas del modo contra la computadora, el juego incluye un **modo para dos jugadores**, de manera que dos personas puedan enfrentarse en el mismo equipo.

El objetivo es aplicar los principios de la programacion orientada a objetos, separando claramente la logica del juego de la interfaz grafica, y demostrar la correcta implementacion recursiva del algoritmo Minimax sobre un juego de suma cero.

## 2. Descripcion del juego

- **Tablero:** cuadrícula de 3 filas por 3 columnas (9 casillas independientes).
- **Fichas:** **X** para el primer jugador y **O** para el segundo.
- **Turnos:** alternancia estricta entre ambos participantes.
- **Fin de partida:** victoria al alinear 3 fichas (horizontal, vertical o diagonal) o empate cuando el tablero se llena sin ganador.

### 2.1 Modos de juego

El juego permite elegir entre dos modos mediante botones de seleccion:

- **Jugador vs Jugador:** dos personas juegan en el mismo equipo alternando sus turnos (X y O).
- **Jugador vs Computadora:** el primer jugador se enfrenta a la IA, que responde con el algoritmo Minimax.

Los nombres de los jugadores son **editables** desde la propia ventana y se reflejan en los mensajes de turno y de resultado. En el modo contra la computadora, el segundo nombre se sustituye por "Computadora".

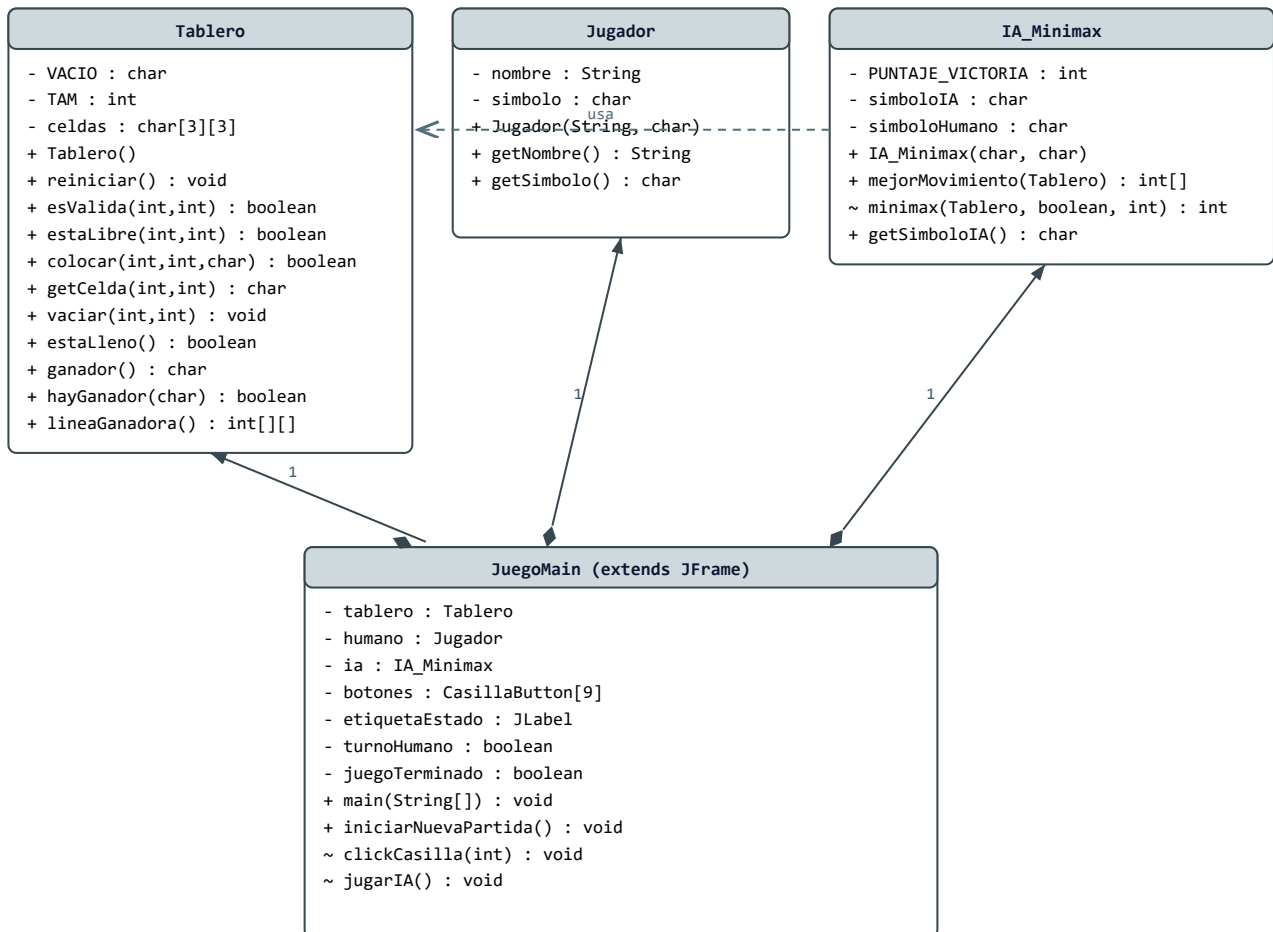
### 3. Requerimientos funcionales cubiertos

---

| Requerimiento             | Como se cumple                                                                                                                          |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Interfaz de usuario       | Ventana Swing con tablero 3×3, barra de estado con el turno y anuncio de ganador/empate, y boton <i>Nueva Partida</i> .                 |
| Validacion de movimientos | El metodo <code>Tablero.estaLibre()</code> impide ocupar casillas usadas o fuera de los limites; las casillas ocupadas se deshabilitan. |
| Logica de IA (Minimax)    | La clase <code>IA_Minimax</code> explora el arbol de decisiones y elige siempre el movimiento optimo: gana o empata, nunca pierde.      |
| Control del juego         | Deteccion automatica de victoria (3 en linea) y empate, con reinicio funcional del tablero.                                             |
| Modos de juego            | Selector entre <b>Jugador vs Jugador</b> y <b>Jugador vs Computadora</b> , con nombres de jugador editables en la interfaz.             |

## 4. Arquitectura y diagrama de clases (UML)

La solución se divide en clases independientes siguiendo los principios de la programación orientada a objetos. La lógica del juego (Tablero, IA\_Minimax) no depende de Swing; la clase JuegoMain actúa como puente entre la lógica y la interfaz gráfica.



Notacion: <filled diamond> composicion → asociacion ---> dependencia

**Notacion:** el rombo relleno indica **composicion** (JuegoMain contiene una instancia de Tablero, Jugador e IA\_Minimax); la flecha punteada indica una **dependencia** (IA\_Minimax utiliza Tablero para explorar el árbol de juego). La clase interna CasillaButton extiende JButton representa cada casilla y pinta su símbolo con color propio incluso deshabilitada.

## 5. Implementacion de la Inteligencia Artificial (Minimax)

El algoritmo **Minimax** es el estandar para juegos de suma cero como el totito. Se construye un **arbol de decisiones** con todos los estados posibles: los nodos donde juega la IA son **maximizadores** (buscan el mayor puntaje) y los nodos donde juega el humano son **minimizadores** (buscan el menor puntaje). El valor de un estado terminal se define asi:

- Victoria de la IA (O):  $+10 - \text{profundidad}$ .
- Victoria del humano (X):  $-10 + \text{profundidad}$ .
- Empate: 0.

Restar la profundidad hace que la IA prefiera ganar lo antes posible y retrasar al maximo una eventual derrota. Como el algoritmo asume que el rival tambien juega de forma optima, la IA **nunca pierde**: gana o empata.

### 5.1 Funcion recursiva minimax

```
int minimax(Tablero tablero, boolean esTurnoIA, int profundidad) {

    // Casos base: estados terminales del arbol de juego
    char ganador = tablero.ganador();
    if (ganador == simboloIA) return PUNTAJE_VICTORIA - profundidad; // gana la IA
    if (ganador == simboloHumano) return profundidad - PUNTAJE_VICTORIA; // gana el humano
    if (tablero.estaLleno()) return 0; // empate

    if (esTurnoIA) {
        // Nivel MAX: la IA elige la jugada con el mayor puntaje
        int mejor = Integer.MIN_VALUE;
        for (cada casilla libre) {
            tablero.colocar(fila, col, simboloIA); // intenta
            int valor = minimax(tablero, false, profundidad + 1);
            tablero.vaciar(fila, col); // deshace (backtracking)
            mejor = Math.max(mejor, valor);
        }
        return mejor;
    } else {
        // Nivel MIN: el humano elige la jugada con el menor puntaje
        int mejor = Integer.MAX_VALUE;
        for (cada casilla libre) {
            tablero.colocar(fila, col, simboloHumano); // intenta
            int valor = minimax(tablero, true, profundidad + 1);
            tablero.vaciar(fila, col); // deshace (backtracking)
            mejor = Math.min(mejor, valor);
        }
        return mejor;
    }
}
```

### 5.2 Seleccion del movimiento

El metodo mejorMovimiento prueba cada casilla libre, evalua la posicion resultante con minimax y devuelve la jugada con el puntaje mas alto para la IA. La jugada se deshace tras evaluarla (*backtracking*) para no alterar el tablero real.

```
public int[] mejorMovimiento(Tablero tablero) {
    int mejorValor = Integer.MIN_VALUE;
    int[] mejor = null;
    for (cada casilla libre) {
        tablero.colocar(fila, col, simboloIA);
        int valor = minimax(tablero, false, 1);    // ahora responde el humano
        tablero.vaciar(fila, col);
        if (valor > mejorValor) { mejorValor = valor; mejor = {fila, col}; }
    }
    return mejor;
}
```

**Complejidad:** el arbol completo del totito tiene a lo sumo  $9! = 362\ 880$  estados. Al podar las ramas que terminan antes (victorias tempranas), el numero de posiciones realmente exploradas es mucho menor, por lo que la IA responde de forma instantanea.

## 6. Casos de prueba

---

Se realizaron dos tipos de prueba: una **verificacion exhaustiva** automatica (recorriendo todas las jugadas posibles del humano) y **capturas de pantalla** de la interfaz real para los escenarios exigidos.

### 6.1 Verificacion exhaustiva (IA imbatible)

Resultado de recorrer todas las ramas del arbol con la IA jugando de forma optima:

```
Partidas evaluadas : 569
Victorias de la IA : 386
Empates           : 183
Victorias humano  : 0
RESULTADO: OK - la IA es imbatible.
```

**El jugador humano no gano en ninguna de las 569 lineas de juego evaluadas.**

### 6.2 Capturas de pantalla

**Modo:** ☐ Jugador vs Jugador ☒ Jugador vs Computadora

Jugador 1 (X):

Computadora (O):

**Tu turno (X)**

|   |   |   |
|---|---|---|
| X | X | O |
|   | O |   |
|   |   |   |

**Nueva Partida**

*Captura 1. Partida en curso: turno del jugador (X) con casillas ocupadas bloqueadas.*

**Modo:** ☐ Jugador vs Jugador ☒ Jugador vs Computadora

Jugador 1 (X):

Computadora (O):

**¡Gano la computadora (O)!**

|   |   |   |
|---|---|---|
| X | X | O |
| X | O |   |
| O |   |   |

**Nueva Partida**

Captura 2. Victoria de la computadora (O) con la línea ganadora resaltada.

**Modo:** ☐ Jugador vs Jugador ☒ Jugador vs Computadora

Jugador 1 (X):

Computadora (O):

**¡Empate! Tablero completo**

|   |   |   |
|---|---|---|
| X | X | O |
| O | O | X |
| X | O | X |

**Nueva Partida**

*Captura 3. Empate: tablero completo sin ganador tras un juego optimo.*

**Modo:** ☒ Jugador vs Jugador ☐ Jugador vs Computadora

Jugador 1 (X):

Jugador 2 (O):

**Turno de Jugador 2 (O)**

|   |   |   |
|---|---|---|
| X | O | X |
| O | X |   |
|   |   |   |

**Nueva Partida**

*Captura 4. Modo Jugador vs Jugador con nombres editables y turno del Jugador 2.*

## 7. Como ejecutar el proyecto

**Requisito:** Java JDK 11 o superior.

```
javac -encoding UTF-8 -d out src\com\umg\totito\*.java
java -cp out com.umg.totito.JuegoMain
```

Tambien se puede ejecutar con doble clic en el archivo ejecutar.bat, que compila y abre el juego automaticamente. El proyecto tambien puede abrirse directamente en cualquier IDE (IntelliJ IDEA, Eclipse o NetBeans) y ejecutar la clase JuegoMain.

## 8. Conclusiones

- Se implemento el algoritmo Minimax de forma recursiva con backtracking, logrando un agente de IA imbatible (verificado en 569 partidas).

- La interfaz Swing cumple con el tablero 3×3, el bloqueo inmediato de casillas, los indicadores de turno/estado y el reinicio de partida.
- El juego integra dos modos (contra la computadora y entre dos personas) con nombres de jugador editables, conservando la IA como requisito.
- La separación en clases (Tablero, Jugador, IA\_Minimax, JuegoMain) demuestra la aplicación de los principios de la programación orientada a objetos.

---

Nombre y firma del estudiante